UNIT - 1

1. Explain the Three-Schema Architecture and discuss its role in achieving data independence.

Introduction :

The Three-Schema Architecture is a conceptual framework used in DBMS to separate the user's view of data from the physical storage of data. It divides the database system into three levels or schemas. This separation helps in achieving data independence, security and reduces complexity.

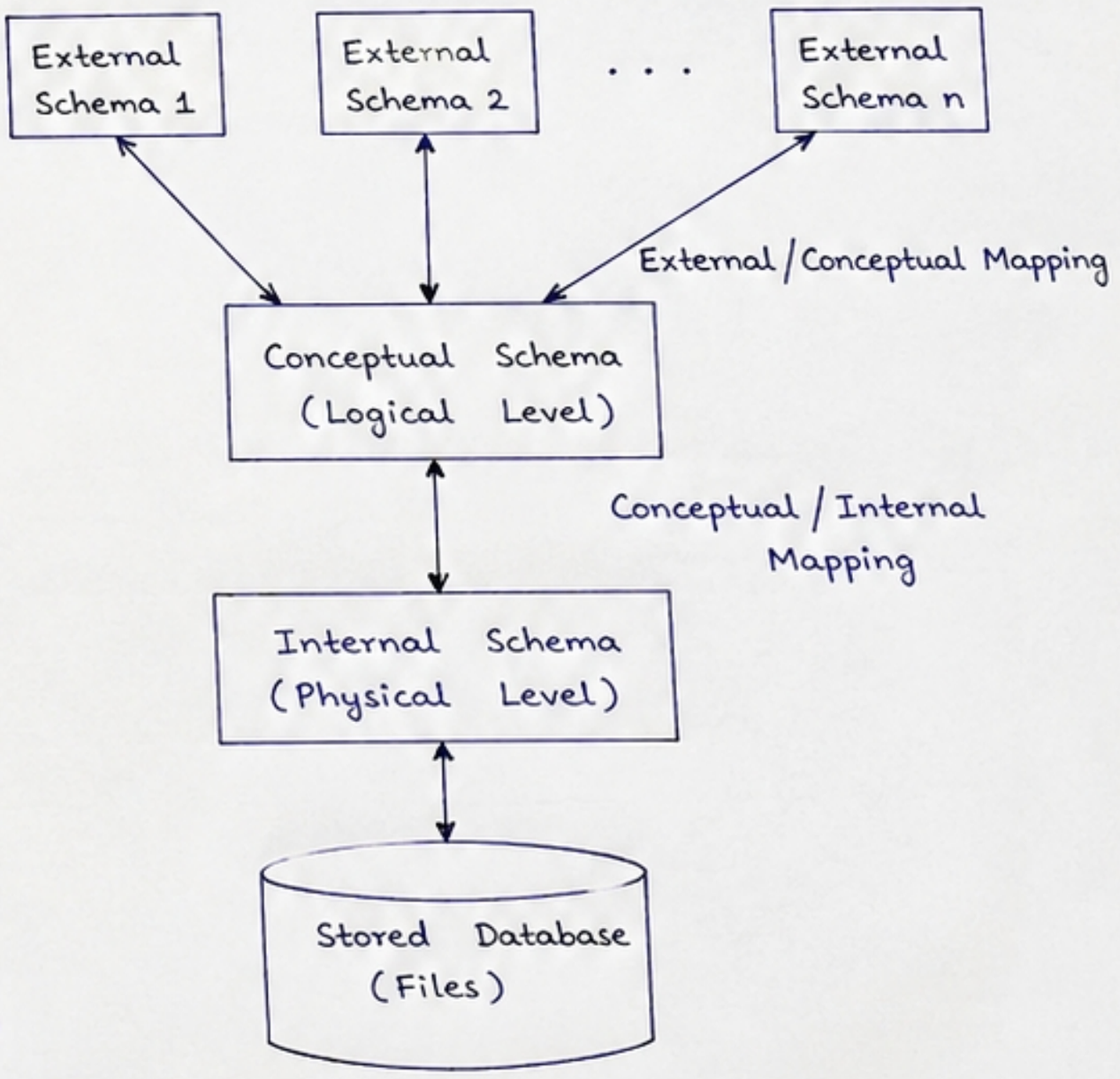
The Three Levels :

① External Level (View Level) :
- It is the user's view of the database.
- Each user or application has its own external schema which shows only the required data.

② Conceptual Level (Logical Level) :
- It is the community view of the entire database.
- It describes what data is stored in the database and the relationships among the data.

③ Internal Level (Physical Level) :
- It is the storage view of the database.
- It describes how the data is actually stored in the secondary storage (files, indexes, etc.).

Diagram :



Data Independence :

Data independence means the ability to change the schema at one level without affecting the schema at the next higher level.

① Logical Data Independence : Changes in the conceptual schema do not affect the external schemas.
Example : Adding a new attribute in the conceptual schema does not affect the user views.

② Physical Data Independence : Changes in the internal schema do not affect the conceptual schema.
Example : Changing the file storage structure or adding an index does not affect the logical structure of the database.

Role in Achieving Data Independence :

- Separation of levels isolates users from the complexity of physical storage.
- Mapping between levels (External / conceptual and conceptual / internal) acts as a bridge and hides changes.
- Hence, DBMS provides flexibility, security, and eases maintenance.

Thus, the Three-Schema Architecture is the backbone of DBMS and plays a vital role in achieving data independence.

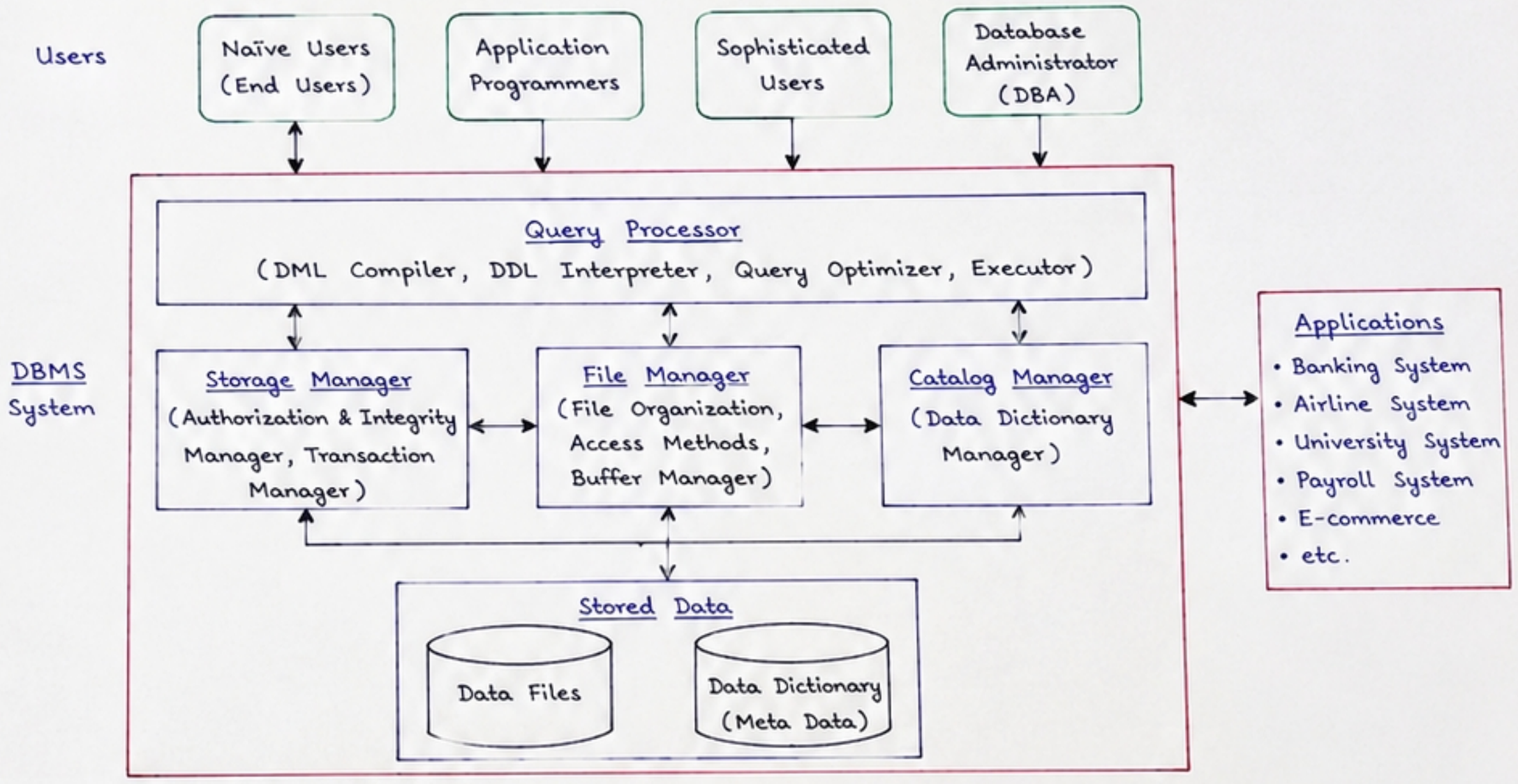
UNIT - 1

2. Describe the architecture of DBMS with a neat diagram and explain the functions of each component.

Answer :

The architecture of a DBMS shows the overall structure of the system, its components and how they interact with each other to manage data efficiently.

Architecture of DBMS :



Functions of each component :

① **Users :** Different types of users interact with the DBMS.
- **Naïve Users :** Use predefined applications (e.g., ATM, ticket booking).
- **Application Programmers :** Write programs that access the database.
- **Sophisticated Users :** Write complex queries (e.g., analysts, engineers).
- **Database Administrator (DBA) :** Manages the entire database system.

② **Query Processor :** It processes the user requests (queries).
- **DML Compiler :** Translates DML queries into low level instructions.
- **DDL Interpreter :** Interprets DDL statements and updates the catalog.
- **Query Optimizer :** Chooses the best execution plan for a query.
- **Executor :** Executes the query and returns the result to the user.

③ **Storage Manager :** It ensures data is stored securely and consistently.
- **Authorization & Integrity Manager :** Checks user permissions and maintains integrity.
- **Transaction Manager :** Manages concurrent transactions and ensures ACID properties.

④ **File Manager :** It is responsible for storing, retrieving and updating data in files.
- **File Organization :** Decides how data is stored on disk.
- **Access Methods :** Provides methods to access data efficiently.
- **Buffer Manager :** Manages main memory buffers for disk data.

⑤ **Catalog Manager (Data Dictionary Manager) :** It stores metadata (data about data). It maintains the data dictionary which contains information about tables, attributes, relationships, constraints, users, etc.

⑥ **Stored Data :** Actual data is stored in data files and metadata is stored in the data dictionary.

⑦ **Applications :** External applications interact with DBMS to perform various real world tasks.

UNIT - 1

3. Explain different types of database users and discuss the responsibilities of the Database Administrator (DBA).

Answer :

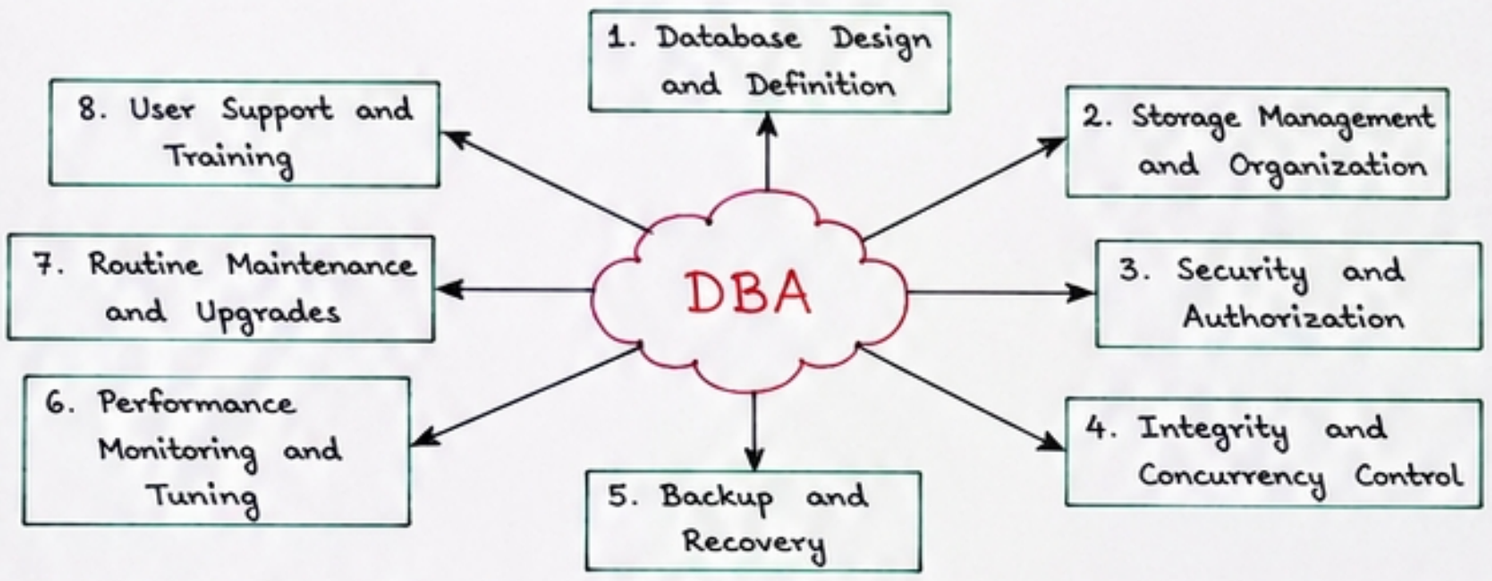
A DBMS is used by several types of users. Each user interacts with the database system in a different way and has different requirements. The main users are classified as follows :

A. Types of Database Users :

Type of User	Description	Example / Role
1. Naïve or Parametric Users	Execute pre-defined (canned) transactions through forms or GUI. They perform routine tasks repeatedly.	Bank teller, Ticket reservation clerk, Data entry operator.
2. Application Programmers	Write application programs that interact with the database using DML/SQL. They develop new database applications.	Software developer who builds payroll application.
3. Sophisticated Users	Use complex queries and analysis tools to extract useful information for decision making.	Data analyst, Engineer, Business analyst.
4. Specialized Users	Develop special-purpose database applications (e.g., CAD, AI, real-time systems).	CAD designers, Scientists, AI applications.
5. Database Administrator (DBA)	Overall control of the database system. Responsible for its design, operation, security and performance.	DBA manages the entire database environment.

B. Responsibilities of Database Administrator (DBA) :

The DBA is the central authority who ensures that the database system works efficiently and securely. The major responsibilities are shown below :



1. Database Design and Definition : Defines the database schema, data types, constraints, views and storage structures.
2. Storage Management and Organization : Decides file organization, indexing, access methods and storage allocation.
3. Security and Authorization : Creates user accounts, grants privileges and ensures data security and confidentiality.
4. Integrity and Concurrency Control : Enforces integrity constraints and controls concurrent access to maintain consistency.
5. Backup and Recovery : Takes regular backups and ensures recovery of database in case of failures.
6. Performance Monitoring and Tuning : Monitors system performance and tunes queries, indexes and storage to improve efficiency.
7. Routine Maintenance and Upgrades : Performs database updates, schema modifications and system upgrades.
8. User Support and Training : Helps users solve problems and provides training on DBMS tools and policies.

Conclusion : Different users interact with DBMS according to their needs, while the DBA ensures the database is secure, reliable, consistent and efficient.

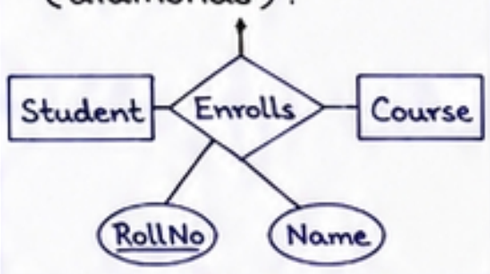
UNIT - 1

4. Compare the Relational, ER, Object-Oriented, and Semi-Structured data models.

Answer :

Different data models are used in DBMS to organize, store and manipulate data. The main data models are Relational, ER, Object-Oriented and Semi-Structured. They are compared below :

Feature	Relational Model	ER Model	Object-Oriented Model	Semi-Structured Model
Basic Concept	Data is represented as tables (relations) consisting of rows (tuples) and columns (attributes).	Data is represented using entities, attributes and relationships. It is a high-level conceptual model.	Data is represented as objects having identity, attributes and methods (behaviour). Supports complex data.	Data is represented in a flexible format where the structure may vary. Uses tags or keys to organize data.

Structure	Structured and tabular.
	ID | Name | Dept
	1 | Ravi | CSE
	2 | Anu | IT
	3 | Hari | ECE
		Graphical representation using entities (rectangles), attributes (ovals) and relationships (diamonds).
		 
			Collection of objects. Each object has attributes and methods.
			Class : Student
			RollNo
			Name
			Dept
			getDetails()
			update()
				Hierarchical or tree-like structure. Data items are nested and self-describing.
				{
				"id": 1,
				"name": "Ravi",
				"dept": "CSE",
				"marks": [85, 90]
				}

Data Independence	Provides high logical and physical data independence.	Conceptual model used for designing DB; mapped to relational model.	Provides high data independence through encapsulation and inheritance.	Provides flexibility but less strict data independence compared to relational model.
Relationships	Represented using primary key and foreign key relationships.	Relationships are explicitly represented using relationship sets (diamonds).	Relationships represented through object references. Supports inheritance.	Relationships are implicit through nesting and embedding.
Query Language	Uses SQL (Structured Query Language).	Uses ER diagram for design. Converted to tables and then SQL is used.	Uses OQL (Object Query Language) or extended SQL.	Uses XML Query Language (XQuery) or JSON querying mechanisms.
Advantages	• Simple and easy to understand. • Supports data integrity, • Widely accepted. • Strong query capability.	• Easy to design at conceptual level. • Clearly represents relationships. • Good for database design.	• Represents real-world objects. • Supports complex data types. • Reusability through inheritance. • Better for multimedia applications.	• Flexible schema. • Handles unstructured or partially structured data. • Easy data exchange (e.g., XML, JSON).
Disadvantages	• Not suitable for complex data (multimedia, CAD, etc.). • Requires table joins for complex relationships.	• Not used directly for implementation. • Needs conversion to relational model. • Can be complex for large systems.	• More complex to implement. • Lack of standardization in querying. • Difficult to optimize compared to relational model.	• Less support for complex transactions. • Hard to enforce constraints. • Querying can be inefficient for large data.

Example	Student (RollNo, Name, Dept, Marks)	ER Diagram of University System.	Class Student {
			RollNo, Name;
			getDetails();
			}
				XML / JSON documents used in web applications.

Conclusion : Relational model is best for structured data and business applications. ER model is used for database design. Object-oriented model is suitable for complex objects and multimedia data. Semi-structured model is useful for flexible and evolving data like XML and JSON.

UNIT - 1

5. Explain the steps involved in database design with a suitable example.

Answer :

Database design is the process of planning and organizing data in a database so that it can be stored efficiently, retrieved easily and maintained effectively. It is usually carried out in the following steps.

Steps in Database Design :

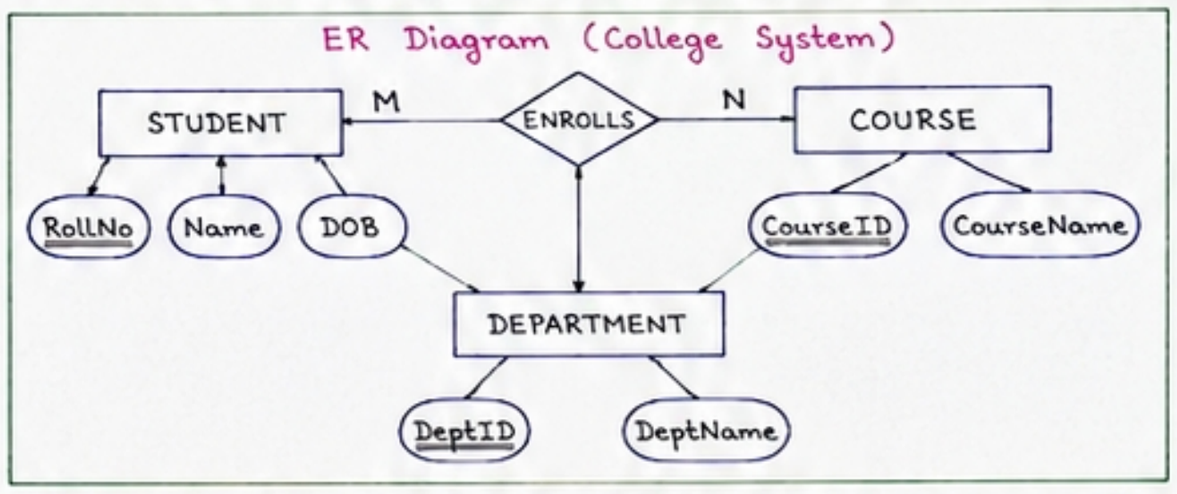
① Requirements Collection and Analysis :

- Collect information about the organization and identify the data to be stored, operations to be performed and constraints.
- Study the existing system and find the problems.

> **Example (College System)**
>
> **Requirements :**
> - Store student details, course details, and marks.
> - Maintain department information.
> - Generate reports like result, topper list etc.

② Conceptual Design :

- Design a high level conceptual schema using ER model.
- Identify entities, attributes and relationships.



③ Logical Design (Mapping) :

- Convert the ER model into a relational schema.
- Determine primary keys, foreign keys and table structures.

> **Relational Schema**
>
> STUDENT (RollNo, Name, DOB, DeptID) [PK: RollNo]
> DEPARTMENT (DeptID, DeptName) [PK: DeptID]
> COURSE (CourseID, CourseName, DeptID) [PK: CourseID]
> ENROLLS (RollNo, CourseID, Mark)
> [PK: RollNo, CourseID]
> [FK: RollNo → STUDENT, CourseID → COURSE]

④ Physical Design :

- Decide storage structures, file organization, indexes, access paths.
- Choose appropriate data types and specify constraints.

> **Example**
>
> - RollNo : VARCHAR(10), Name : VARCHAR(30)
> - DOB : DATE, Mark : INT
> - Index on RollNo for fast search.
> - Tables stored using B+ Tree index.

⑤ Implementation :

- Create the database schema using DDL commands.
- Create tables, relationships, constraints, views, indexes etc.

> **Example (SQL)**

```
CREATE TABLE STUDENT (RollNo VARCHAR(10) PRIMARY KEY,
                      Name VARCHAR(30), DOB DATE, DeptID INT,
                      FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID));
CREATE TABLE COURSE (CourseID VARCHAR(10) PRIMARY KEY,
                     CourseName VARCHAR(30), DeptID INT,
                     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID));
CREATE TABLE ENROLLS (RollNo VARCHAR(10), CourseID VARCHAR(10),
                      Mark INT, PRIMARY KEY (RollNo, CourseID),
                      FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo),
                      FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID));
```

⑥ Testing and Evaluation :

- Test the database for correctness, efficiency and security.
- Validate the data and check for performance.

> **Example**
>
> - Test queries for different conditions.
> - Check data integrity and constraints.
> - Evaluate response time and optimize if needed.

⑦ Maintenance :

- Modify the database as per changing requirements.
- Backup, update data and improve performance.

> **Example**
>
> - Add new fields (e.g., Email, Phone).
> - Update data, fix errors and improve security.
> - Regular backups and recovery.

Conclusion : Thus, database design is a systematic process starting from requirement analysis to maintenance. A well designed database ensures data integrity, redundancy control and efficient data management.

UNIT - 1

6. Discuss Parallel Databases and Distributed Databases.

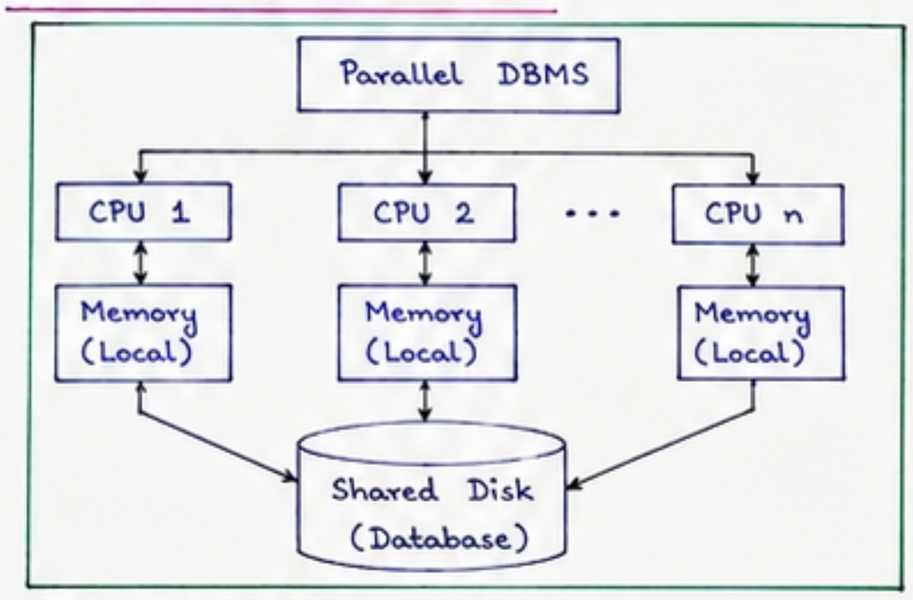
Answer :

With the growth of data and users, traditional DBMS became insufficient. To improve performance, reliability and scalability, databases are designed as Parallel Databases and Distributed Databases.

A. PARALLEL DATABASES

- A parallel database is a database system in which multiple processors (CPU's) work simultaneously on different parts of the same database.
- The system is tightly coupled and processors share the same memory and disk.
- Goal : To increase speed (throughput), improve reliability and handle large volume of data.
- Types of Parallelism :
  - → Inter-query parallelism
  - → Intra-query parallelism

Architecture (Parallel DBMS) :



Advantages :

1. High performance due to parallel execution,
2. Better reliability and availability.
3. Efficient handling of large databases.
4. Economies of scale.

Disadvantages :

1. High cost of hardware and software.
2. Complex design and management.
3. Requires special parallel DBMS software.

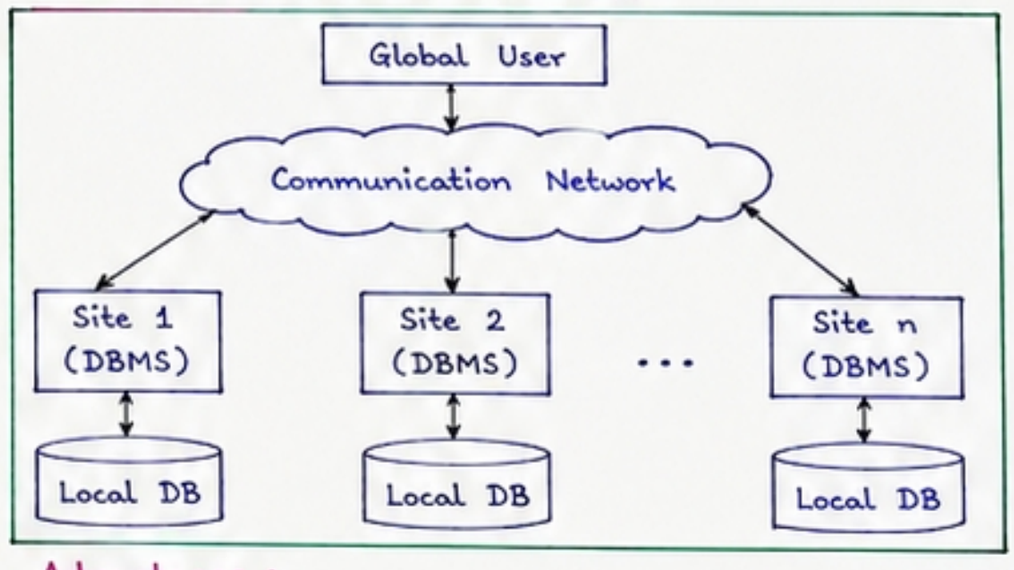
B. DISTRIBUTED DATABASES

- A distributed database is a collection of logically related data stored at multiple physically distributed sites connected through a network.
- Each site has its own database and DBMS but users can access the data as if it is a single database.
- Goal : To provide data availability, reliability, local autonomy and better performance.

Types of Distribution :

- → Homogeneous distribution (same DBMS software at all sites)
- → Heterogeneous distribution (different DBMS software at sites)

Architecture (Distributed DBMS) :



Advantages :

1. Data is available at multiple locations.
2. High reliability and fault tolerance.
3. Local autonomy and flexibility.
4. Easy scalability and load balancing.

Disadvantages :

1. Complex design and management.
2. High communication cost.
3. Data consistency and security difficult to maintain.

Comparison :

Feature	Parallel Databases	Distributed Databases
Meaning	Multiple processors work on the same database to improve speed.	Data is stored at multiple sites connected through a network.
Nature	Tightly coupled system.	Loosely coupled system.
Data Location	Data is stored at a single site (shared disk).	Data is stored at multiple sites.
Main Objective	Increase performance by parallel processing.	Increase availability, reliability and local autonomy.
Communication	Through shared memory and bus.	Through computer network.
Example	Oracle Parallel Server, DB2 Parallel Edition	Google Spanner, Amazon Aurora Global Database

Conclusion : Parallel databases focus on speed using multiple processors at one site, whereas distributed databases focus on data availability and reliability across multiple sites.

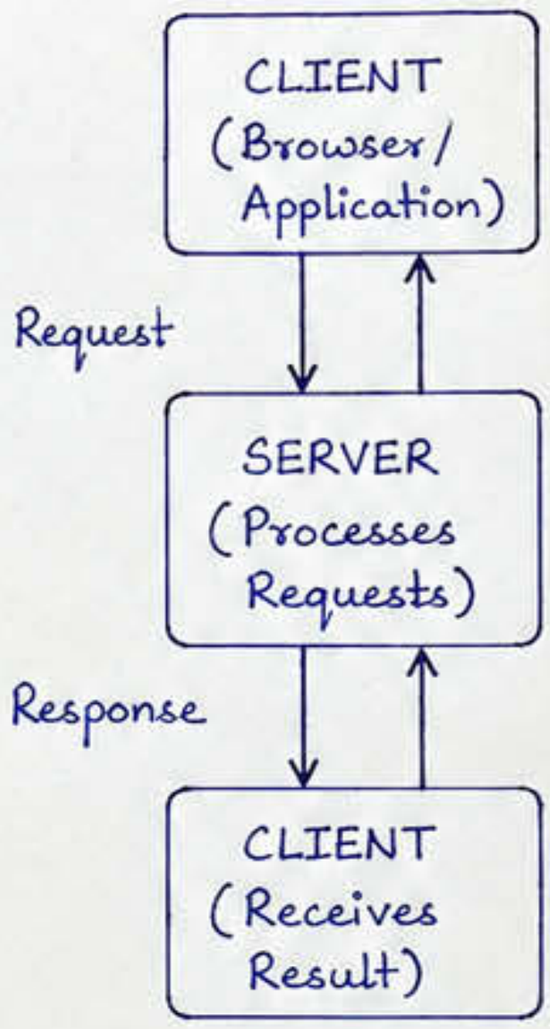
Q. Explain Client-Server Architecture with neat diagrams.

Client-Server Architecture

Client-Server Architecture is a network architecture in which a client requests services or resources, and a server processes the request and sends the response.

The client is generally a user's device or application, while the server is a powerful computer or software that stores data and provides services.

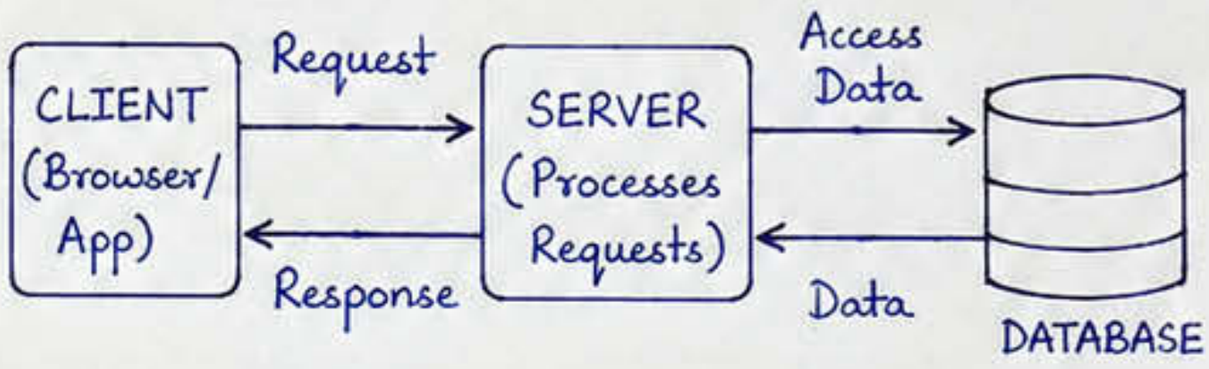
Diagram 1 : Basic Client-Server Interaction



Working of Client-Server Architecture

1. The client sends a request to the server.
2. The server receives and processes the request.
3. The server may access a database if required.
4. The server prepares the requested information.
5. The response is sent back to the client.

Diagram 2 : Client-Server Architecture with Database



Advantages

- Centralized management of data and resources.
- Easy data sharing among multiple clients.
- Better security and access control.
- Easy maintenance and updates.
- Multiple clients can communicate with one server.

Disadvantages

- Server failure can affect all connected clients.
- A high number of requests may slow down the server.
- Requires proper network infrastructure.
- Setup and maintenance can be costly.

Conclusion

Client-Server Architecture is widely used in modern applications such as websites, banking systems, email services, online shopping applications, and cloud services. It provides an efficient way to manage resources by separating the service requester (client) from the service provider (server).